# Simple Guide: Distributed Food Ordering

Kafka (KRaft) + Spark Batch · Big Data 2026 (UE24CS343AB2) · Team of 4: Member 1 = VM1, Member 2 = VM2, Member 3 = VM3, Member 4 = VM4


| Section | What's in it |
|---|---|
| 1. Big picture | What the assignment does, the 4 machines, the rules |
| 2. Demo day overview | Order of turns, what the evaluator checks, how to copy commands |
| 3. Member 1 (VM1) | Producer 1, Consumer 2, Spark Job 2: explanation + demo commands |
| 4. Member 2 (VM2) | Broker 1 + controller: explanation + demo commands |
| 5. Member 3 (VM3) | Broker 2: explanation + demo commands |
| 6. Member 4 (VM4) | Consumer 1, Spark Job 1, Producer 2: explanation + demo commands |
| 7-10 | Failure test explained, key words, viva questions, numbers to remember |


## 1. The whole assignment in one minute

A food delivery app has a file of **62 orders** (`order.jsonl`). Some of them are junk: broken records, cancelled orders, bad timestamps, duplicates. We have to:

- **Send** the raw orders through Kafka (VM1 -> VM4).
- **Clean** them and add delivery numbers with Spark (VM4). 62 raw -> **44 good orders**.
- **Send** the cleaned orders back through Kafka (VM4 -> VM1).
- **Summarise** them with Spark (VM1) into two reports: one per restaurant, one per city.
- **Prove** that Kafka keeps working when one broker (VM3) is switched off.

**Analogy.** Kafka is a **post office** that stores letters (messages) in labelled mailboxes (*topics*). It keeps two copies of every letter, one in each branch (VM2 and VM3), so if one branch closes the letters are still there. Spark is the **calculator** each side uses on its own desk after collecting the mail.

### The 4 machines

| Member / VM | Role | What it does in plain words |
|---|---|---|
| Member 1 / VM1 | Producer 1, Consumer 2, Spark Job 2 | Posts the raw orders into Kafka. Later collects the cleaned orders and makes the two final CSV reports. |
| Member 2 / VM2 | Kafka broker 1 + controller | A Kafka server, and also the **boss** of the cluster: it knows which servers are alive and who is in charge of each piece of data. |
| Member 3 / VM3 | Kafka broker 2 | The second Kafka server. Holds a backup copy of everything. This is the one we switch off in the failure test. |
| Member 4 / VM4 | Consumer 1, Spark Job 1, Producer 2 | Collects the raw orders, cleans them and adds metrics with Spark, then posts the cleaned orders back into Kafka. |

### The flow, step by step

```
STAGE 1                                    STAGE 2
VM1  producer1.py                          VM4  producer2.py
      |  topic: orders  (62 msgs)                |  topic: order_metrics (44 msgs)
      v                                          v
VM4  consumer1.py -> orders_landed.jsonl   VM1  consumer2.py -> order_metrics_landed.jsonl
      v                                          v
VM4  spark_job1.py -> order_metrics.jsonl  VM1  spark_job2.py -> restaurant_summary.csv
      (clean + enrich, 62 -> 44)                                  -> city_summary.csv

Both topics live on the Kafka cluster = VM2 + VM3 (connected over ZeroTier)
```

```
Each topic: 2 partitions, replication factor 2
```

## Rules from the spec (the things examiners check)

- Spark never reads Kafka directly. A consumer first saves the data to a local `.jsonl` file; Spark reads that file.
- No HDFS. Everything is on the normal local disk (`outputs/`).
- VM4 must not copy its file to VM1. It must go through Kafka (topic `order_metrics`).
- Both topics: **2 partitions, replication factor 2**, same cluster.
- Failure test: stop **VM3**, never VM2 (VM2 is the only boss).
- Exact duplicates counted once in both Spark jobs. All calculated numbers rounded to 2 decimals. Nothing hardcoded.

# 2. Demo day overview

Run the steps **in number order**. Each member runs only their own steps (in their section below) and says **"done"** when they finish, then the next member starts. The same commands are in `DEMO_RUNBOOK.md` in the repo.

**Copying commands:** copy one grey box at a time and paste it into the terminal. Every line in a box is a real command. A line ending in `\` continues on the next line, so copy both lines together. Replace `VM2_ZEROTIER_IP` and `VM3_ZEROTIER_IP` with the real IPs.

## Order of turns

| Steps | Member | What happens |
|---|---|---|
| 1 | Member 2 | Start broker 1 + controller |
| 2 | Member 3 | Start broker 2 |
| 3-4 | Member 2 | Check both brokers, create fresh topics |
| 5-6 | Members 1 + 4 | Clear old outputs, check brokers are reachable |
| 7 | Member 1 | Producer 1 |
| 8-10 | Member 4 | Consumer 1, Spark Job 1, Producer 2 |
| 11-13 | Member 1 | Consumer 2, Spark Job 2, verify (graded outputs) |
| 14 | Member 2 | Show topics before the failure |
| 15 | Member 3 | Stop broker 2 |
| 16 | Member 2 | Show topics after the failure |
| 17 | Member 1 | Producer 1 with VM3 down |
| 18-20 | Member 4 | Consumer 1, Spark Job 1, Producer 2 with VM3 down |
| 21-23 | Member 1 | Consumer 2, Spark Job 2, compare results |
| 24 | Member 3 | Restart broker 2 |
| 25 | Member 2 | Show VM3 rejoined the ISR |
| 26 | Member 1 | Producer 1 |
| 27-29 | Member 4 | Consumer 1, Spark Job 1, Producer 2 |
| 30-32 | Member 1 | Consumer 2, Spark Job 2, compare results (demo finished) |

## What the evaluator checks

| TA checks | Steps | Who shows it | Evidence on screen |
|---|---|---|---|
| Cluster is up, 2 brokers | 3 | Member 2 | `id: 1` and `id: 2` |
| Topic config | 4, 14 | Member 2 | PartitionCount 2, ReplicationFactor 2, Isr 1,2 |
| Pipeline output (accuracy) | 7-13 | Members 1 + 4 | 62 -> 44 -> 6 + 4 rows; verify says OK |
| ISR / leaders after VM3 stops | 15-16 | Members 3 + 2 | Leader 1, Isr 1 everywhere; 4 under-replicated |
| Pipeline continues with VM3 down | 17-23 | Members 1 + 4 | 124 landed, 44 enriched, 88 landed, SAME RESULTS |
| VM3 rejoins the ISR | 24-25 | Members 3 + 2 | Isr back to 1,2; under-replicated list empty |
| Pipeline after recovery | 26-32 | Members 1 + 4 | 186, 44, 132, SAME RESULTS |

## Before you start (everyone)

- `git pull` so every VM has the same code. ZeroTier connected; you can `ping` VM2's ZeroTier IP.

- Run your **setup** box (step 0) in **every** terminal you'll use.
- Never stop VM2. Don't edit `order.jsonl`.
- Spark's `WARN` lines are normal. Only a `Traceback` means a failure.
- Keep each command's output on screen until the evaluator has seen it.

# 3. Member 1 (VM1): Producer 1, Consumer 2, Spark Job 2

Member 1 runs three programs: **producer1.py** at the start, then later **consumer2.py** and **spark_job2.py** at the end. Two shared helper files sit behind them: `publish.py` and `landing.py`.

## 3.1 producer1.py: post the raw orders

Reads `order.jsonl` line by line and sends each line to topic **orders**. All of the real work is in the shared helper `publish.py`.

- **Sends every line as-is, junk included.** The producer only delivers; cleaning is Spark Job 1's job. If both did some cleaning, the rules would be split across two places.
- **No key.** To get a key (order_id) it would have to read the JSON, and some lines are broken. Without a key Kafka spreads messages randomly over the 2 partitions (e.g. 31 / 31, and the split changes each run).
- **acks="all"**: Kafka only says "received" once *every* in-sync copy has the message. This is the safest setting.
- **retries=10**: if something hiccups (e.g. VM3 goes down mid-send), it tries again automatically.
- **linger_ms=10**: waits up to 10 ms to bundle messages together, so fewer network trips.
- It waits for confirmation of every message (`flush()` + `f.get()`), so the printed "62 sent" means 62 *confirmed*, along with which partition each one landed in.

## 3.2 consumer2.py + landing.py: collect the cleaned orders

Reads *everything* from topic **order_metrics** and saves it to `order_metrics_landed.jsonl` (44 lines). Member 4's consumer1.py is the same code with topic **orders**.

- **Reads all partitions itself** (`assign()`, no consumer group). A consumer group remembers where it stopped, so a second run would get nothing. We want the *whole* topic every time.
- **Always starts from the beginning** (`seek_to_beginning`, offset 0).
- **Knows when to stop.** Normal consumers run forever. Ours stops once it has reached the last message of every partition *and* nothing new has arrived for 10 s (`--idle-timeout`).
- **Can start early.** `--wait-timeout` (default 120 s) lets it wait for the producer.
- **Overwrites the file** (`"w"` mode), so running it twice doesn't double the data.
- Prints how many records came from each partition: **16 from partition 0, 28 from partition 1**.

## 3.3 spark_job2.py: make the two reports

Reads `order_metrics_landed.jsonl` and writes **restaurant_summary.csv** (6 rows) and **city_summary.csv** (4 rows).

| Step | What happens | Why |
|---|---|---|
| 1. Read | Read the file with a fixed schema (types given up front) | Types are guaranteed and the file is read only once |
| 2. Dedup | `dropDuplicates` on all 15 columns | Spec: exact duplicates count once. Removes 0 here because Job 1 already did it, but it protects against a producer being run twice |
| 3. Group | groupBy restaurant_id, then groupBy city | The two analytics the spec asks for |
| 4. Compute | count, averages, delayed count (rows where status = DELAYED), delay_rate_pct = 100 x delayed / count | delay_rate is worked out from the exact counts, and rounded only once at the end |
| 5. Round | `F.round(..., 2)` on every calculated number | Spec rule |

| Step | What happens | Why |
|---|---|---|
| 6. Sort + write | Sort the rows, collect them (just 10 rows) and write with Python's csv module | Spark's own writer makes a *folder* of part-files, but the spec wants one file with an exact name |

`.cache()` keeps the deduplicated data in memory because it is used by two reports. Without it Spark would recompute it for each one.

## 3.4 Final outputs (know these by heart)

| restaurant | orders | avg amount | avg prep | avg delivery | delayed | delay % |
|---|---|---|---|---|---|---|
| R01 | 8 | 548.5 | 13.13 | 22.0 | 4 | 50.0 |
| R02 | 8 | 539.25 | 13.38 | 27.0 | 8 | 100.0 |
| R03 | 7 | 664.0 | 13.86 | 32.0 | 4 | 57.14 |
| R04 | 7 | 639.57 | 12.14 | 37.0 | 7 | 100.0 |
| R05 | 7 | 615.14 | 12.0 | 42.0 | 3 | 42.86 |
| R06 | 7 | 712.14 | 13.43 | 47.0 | 7 | 100.0 |

| city | orders | avg amount | avg delivery | delayed | delay % |
|---|---|---|---|---|---|
| Bengaluru (R01+R02) | 16 | 543.88 | 24.5 | 12 | 75.0 |
| Hyderabad (R06) | 7 | 712.14 | 47.0 | 7 | 100.0 |
| Mumbai (R03+R04) | 14 | 651.79 | 34.5 | 11 | 78.57 |
| Pune (R05) | 7 | 615.14 | 42.0 | 3 | 42.86 |

**Easy cross-check:** each restaurant is in exactly one city, so Pune = R05 and Hyderabad = R06. Bengaluru = R01 + R02: 8+8 = 16 orders, 4+8 = 12 delayed, 12/16 = 75%. Order counts add up to 44; delayed orders add up to 33 of 44. Example: R05 delay % = 100 x 3 / 7 = 42.857 -> **42.86**.

## 3.5 Demo commands

**Your steps:** 0, 5, 6, 7, 11, 12, 13, 17, 21, 22, 23, 26, 30, 31, 32. You start and finish every run.
**Show the evaluator:** 62 published; 44 landed (16 / 28); 6 + 4 rows; verify OK; both CSVs; `SAME RESULTS` after the failure and after recovery.
**Be ready to explain:** why Producer 1 has no key; why the consumer reads every partition from offset 0 and stops by itself; how Job 2 dedupes, groups and rounds; why the counts grow to 88 and 132 while the CSVs stay the same.

### Step 0 · Setup (every terminal)

```
cd /mnt/c/Users/Aryan/Desktop/BD-Assignment
source .venv-wsl/bin/activate
export KAFKA_HOME=~/kafka
export JAVA_HOME=/usr/lib/jvm/java-21-openjdk-amd64
unset SPARK_HOME HADOOP_HOME HADOOP_CONF_DIR PYSPARK_PYTHON PYSPARK_DRIVER_PYTHON
export B1=VM2_ZEROTIER_IP:9092
export B2=VM3_ZEROTIER_IP:9092
export BS=$B1
export BOOTSTRAP=$B1,$B2
export KAFKA_BOOTSTRAP_SERVERS=$B1,$B2
```

**Expect:** the prompt starts with `(.venv-wsl)`.

### Step 5 · Clear old outputs start after: Member 2, step 4

```
rm -rf outputs/*.jsonl outputs/*.csv outputs/failover_test outputs/recovery_test
mkdir -p outputs/failover_test outputs/recovery_test
```

### Step 6 · Check the brokers are reachable start after: step 5

```
nc -zv VM2_ZEROTIER_IP 9092
nc -zv VM3_ZEROTIER_IP 9092
```

**Expect:** `succeeded` twice.

### Step 7 · Producer 1 start after: step 6

```
python producer/producer1.py --input order.jsonl
```

**Expect:** `published 62 records`.

### Step 11 · Consumer 2 start after: Member 4, step 10

```
python consumer/consumer2.py --output outputs/order_metrics_landed.jsonl
```

**Expect:** `landed 44 records`, partition 0: 16, partition 1: 28.

### Step 12 · Spark Job 2 start after: step 11

```
python spark/spark_job2.py --input outputs/order_metrics_landed.jsonl \
    --output-dir outputs
```

**Expect:** `6 rows -> restaurant_summary.csv` and `4 rows -> city_summary.csv`.

### Step 13 · Verify and show the results start after: step 12

```
python tests/verify_outputs.py --raw order.jsonl \
    --metrics outputs/order_metrics_landed.jsonl
cat outputs/restaurant_summary.csv outputs/city_summary.csv
```

**Expect:** `OK: 44 enriched records ...` and the two tables from section 3.4.

### Step 17 · Producer 1 with VM3 down start after: Member 2, step 16

```
python producer/producer1.py --bootstrap-servers $B1 --input order.jsonl
```

**Expect:** `published 62 records`, so writes still work.

### Step 21 · Consumer 2 with VM3 down start after: Member 4, step 20

```
python consumer/consumer2.py --bootstrap-servers $B1 \
    --output outputs/failover_test/order_metrics_landed.jsonl
```

**Expect:** `landed 88 records` (44 old + 44 new).

### Step 22 · Spark Job 2 with VM3 down start after: step 21

```
python spark/spark_job2.py --input outputs/failover_test/order_metrics_landed.jsonl \
    --output-dir outputs/failover_test
```

**Expect:** `exact duplicates removed: 44`, then 6 + 4 rows.

### Step 23 · Show the results are unchanged start after: step 22

```
diff outputs/restaurant_summary.csv outputs/failover_test/restaurant_summary.csv
diff outputs/city_summary.csv outputs/failover_test/city_summary.csv \
    && echo "SAME RESULTS"
```

**Expect:** `SAME RESULTS`. Tell Member 3 to restart the broker.

### Step 26 · Producer 1 after recovery start after: Member 2, step 25

```
python producer/producer1.py --input order.jsonl
```

**Expect:** `published 62 records`.

### Step 30 · Consumer 2 after recovery start after: Member 4, step 29

```
python consumer/consumer2.py --output outputs/recovery_test/order_metrics_landed.jsonl
```

**Expect:** `landed 132 records`.

### Step 31 · Spark Job 2 after recovery start after: step 30

```
python spark/spark_job2.py --input outputs/recovery_test/order_metrics_landed.jsonl \
    --output-dir outputs/recovery_test
```

**Expect:** 6 + 4 rows.

### Step 32 · Show the results are unchanged start after: step 31

```
diff outputs/restaurant_summary.csv outputs/recovery_test/restaurant_summary.csv
diff outputs/city_summary.csv outputs/recovery_test/city_summary.csv \
    && echo "SAME RESULTS"
```

**Expect:** `SAME RESULTS`. **Demo finished.**

# 4. Member 2 (VM2): Broker 1 + KRaft controller (the boss)

- Runs Kafka with `process.roles=broker,controller`, `node.id=1`.
- **KRaft** = Kafka manages itself, with no separate ZooKeeper. The controller keeps the cluster's "register" (which brokers are alive, which topics exist, who leads each partition).
- Port **9092** = normal Kafka traffic, port **9093** = controller traffic (only VM2 has it).
- Generates the **cluster ID** once; VM3 must use the same ID to join.
- `min.insync.replicas=1`: writes still succeed with only one copy alive (needed while VM3 is off).
- `auto.create.topics.enable=false`: topics are only created on purpose, so a typo can't create a wrong topic.
- In the demo, Member 2 shows the cluster state: both brokers, the topic config, and the ISR before, during and after the failure (see section 7 for how to read it).

## 4.1 Demo commands

**Your steps:** 0, 1, 3, 4, 14, 16, 25. You show the cluster state.
**Show the evaluator:** both broker ids; topics with 2 partitions and RF 2; Isr 1,2 before, then Leader 1 / Isr 1 after VM3 stops, then Isr 1,2 again after it restarts.
**Be ready to explain:** KRaft and what the controller does; node.id, listeners vs advertised.listeners, ports 9092/9093; cluster ID; min.insync.replicas=1; Leader vs Replicas vs Isr; why VM2 can't be stopped.

### Step 0 · Setup (every terminal)

```
cd ~/BD-Assignment
export KAFKA_HOME=~/kafka
export B1=VM2_ZEROTIER_IP:9092
export B2=VM3_ZEROTIER_IP:9092
export BS=$B1
export BOOTSTRAP=$B1,$B2
export KAFKA_BOOTSTRAP_SERVERS=$B1,$B2
```

### Step 1 · Start broker 1 and the controller

start after: step 0

```
$KAFKA_HOME/bin/kafka-server-start.sh -daemon ~/broker1.properties
```

**Expect:** no output. Tell Member 3 to start.

### Step 3 · Check both brokers joined

start after: Member 3, step 2

```
$KAFKA_HOME/bin/kafka-broker-api-versions.sh --bootstrap-server $B1 | grep "id:"
```

**Expect:** two lines, `id: 1` and `id: 2`.

### Step 4 · Create fresh topics

start after: step 3

```
./kafka/topics.sh delete
sleep 5
./kafka/topics.sh create
```

**Expect:** both topics with `PartitionCount: 2`, `ReplicationFactor: 2`, Isr 1,2. Tell Members 1 and 4 to start.

### Step 14 · Show the topics before the failure start after: Member 1, step 13

```
./kafka/topics.sh describe
$KAFKA_HOME/bin/kafka-topics.sh --bootstrap-server $B1 \
    --describe --under-replicated-partitions
```

**Expect:** Isr lists 1 and 2 on every partition; the second command prints nothing.

### Step 16 · Show the topics after the failure start after: Member 3, step 15

```
sleep 15
BOOTSTRAP=$B1 ./kafka/topics.sh describe
$KAFKA_HOME/bin/kafka-topics.sh --bootstrap-server $B1 \
    --describe --under-replicated-partitions
```

**Expect:** `Leader: 1` and `Isr: 1` on every partition; the second command lists all 4 partitions. Tell Member 1 to start.

### Step 25 · Show VM3 rejoined start after: Member 3, step 24

```
sleep 15
./kafka/topics.sh describe
$KAFKA_HOME/bin/kafka-topics.sh --bootstrap-server $B1 \
    --describe --under-replicated-partitions
```

**Expect:** Isr lists 1 and 2 again; the second command prints nothing. Leader may stay 1 for a few minutes; that's normal. Tell Member 1 to start.

# 5. Member 3 (VM3): Broker 2 (the backup)

- Runs Kafka with `process.roles=broker` only, `node.id=2`.
- Points at VM2:9093 to join and is formatted with **VM2's cluster ID** (it must not create its own cluster).
- Holds the second copy of every partition. In the failure test, **this is the machine that gets stopped**.
- When it stops, VM2's controller notices the missing heartbeats, removes it from the ISR and makes broker 1 the leader everywhere. When it restarts, it copies what it missed and rejoins the ISR.

## 5.1 Demo commands

**Your steps:** 0, 2, 15, 24. Your broker is the one that fails.
**Show the evaluator:** the broker stopping and starting again. Then point to Member 2's describe output, where Isr shrinks to 1 and grows back to 1,2.
**Be ready to explain:** what happens step by step when VM3 stops (heartbeats stop, it's fenced, leadership moves to broker 1); how it catches up and rejoins the ISR; why VM3 uses VM2's cluster ID; what RF 2 protects against.

### Step 0 · Setup (every terminal)

```
cd ~/BD-Assignment
export KAFKA_HOME=~/kafka
```

### Step 2 · Start broker 2 start after: Member 2, step 1

```
$KAFKA_HOME/bin/kafka-server-start.sh -daemon ~/broker2.properties
```

**Expect:** no output. Tell Member 2.

### Step 15 · Stop broker 2 (the failure) start after: Member 2, step 14

```
$KAFKA_HOME/bin/kafka-server-stop.sh
```

**Expect:** the broker stops. Tell Member 2.

### Step 24 · Restart broker 2 (recovery) start after: Member 1, step 23

```
$KAFKA_HOME/bin/kafka-server-start.sh -daemon ~/broker2.properties
```

**Expect:** no output. Tell Member 2.

# 6. Member 4 (VM4): Consumer 1, Spark Job 1, Producer 2 (the cleaning step)

- **consumer1.py**: same code as consumer2, but reads topic **orders** -> `orders_landed.jsonl` (62 lines).
- **spark_job1.py**: the heart of the project. It **throws away bad records**, **removes duplicates**, and **adds 7 new columns**. Output: `order_metrics.jsonl` (44 lines).
- **producer2.py**: posts those 44 lines to topic **order_metrics**, **keyed by order_id**. The same key always goes to the same partition, so the split is always **16 / 28**.

**Cleaning funnel: 62 -> 48 -> 44**

| Thrown out because... | Count |
|---|---|
| status is not DELIVERED (T04X0001-6: cancelled, pending, failed, refunded) | 6 |
| no order_id (2 "invalid_order_record" lines) | 2 |
| missing fields ({"order_id":"BROKEN"} and a T040002 copy without a city) | 2 |
| bad timestamp (T040005 "bad", T040003 "not-a-timestamp") | 2 |
| distance_km <= 0 (T040001: -2) and order_amount <= 0 (T040004: 0) | 2 |
| **Total dropped -> 48 valid** | **14** |
| Exact duplicates removed (T040006, T040017, T040028, T040039) -> **44** | 4 |

**New columns Spark Job 1 adds** (all in minutes, rounded to 2 decimals):

| Column | Formula |
|---|---|
| preparation_time_min | picked_up - accepted |
| delivery_time_min | delivered - picked_up |
| fulfillment_time_min | delivered - order_timestamp |
| delivery_speed_kmph | distance_km / (delivery_time_min / 60) |
| delay_min | max(0, delivered - expected) |
| delivery_status | ON_TIME if delay = 0, else DELAYED |
| delay_category | 0 ON_TIME \| up to 15 SLIGHT \| up to 30 MODERATE \| over 30 SEVERE |

**Midnight trick:** it uses full date+time, not just the time of day. T040023 picked up 23:20 and delivered 00:02 the next day = **42 min**, not -1398. The time zone is set to UTC so daylight saving can't shift anything.

## 6.1 Demo commands

**Your steps:** 0, 5, 6, 8, 9, 10, 18, 19, 20, 27, 28, 29. You do the middle of every run.
**Show the evaluator:** 62 landed; the drop report (14 dropped, 48 valid, 4 duplicates, 44 written); 44 published, split 16 / 28; still 44 enriched with VM3 down.
**Be ready to explain:** each cleaning rule and the 62 -> 48 -> 44 funnel; the 7 new columns and the midnight case (T040023 = 42 min); why Producer 2 is keyed by order_id; why Job 1 still gives 44 from 124 lines.

### Step 0 · Setup (every terminal)

```
cd ~/BD-Assignment
source .venv/bin/activate
export KAFKA_HOME=~/kafka
export B1=VM2_ZEROTIER_IP:9092
export B2=VM3_ZEROTIER_IP:9092
export BS=$B1
export BOOTSTRAP=$B1,$B2
export KAFKA_BOOTSTRAP_SERVERS=$B1,$B2
```

**Expect:** the prompt starts with `(.venv)`. If Spark later fails with `'JavaPackage' object is not callable`, run `unset SPARK_HOME HADOOP_HOME HADOOP_CONF_DIR PYSPARK_PYTHON PYSPARK_DRIVER_PYTHON`.

### Step 5 · Clear old outputs start after: Member 2, step 4

```
rm -rf outputs/*.jsonl outputs/*.csv outputs/failover_test outputs/recovery_test
mkdir -p outputs/failover_test outputs/recovery_test
```

### Step 6 · Check the brokers are reachable start after: step 5

```
nc -zv VM2_ZEROTIER_IP 9092
nc -zv VM3_ZEROTIER_IP 9092
```

**Expect:** `succeeded` twice.

### Step 8 · Consumer 1 start after: Member 1, step 7

```
python consumer/consumer1.py --output outputs/orders_landed.jsonl
```

**Expect:** `landed 62 records`.

### Step 9 · Spark Job 1 start after: step 8

```
python spark/spark_job1.py --input outputs/orders_landed.jsonl \
    --output outputs/order_metrics.jsonl
```

**Expect:** `valid records: 48`, `exact duplicates removed: 4`, `enriched records written: 44`.

### Step 10 · Producer 2 start after: step 9

```
python producer/producer2.py --input outputs/order_metrics.jsonl
```

**Expect:** `published 44 records`, partition 0: 16, partition 1: 28. Tell Member 1.

### Step 18 · Consumer 1 with VM3 down start after: Member 1, step 17

```
python consumer/consumer1.py --bootstrap-servers $B1 \
    --output outputs/failover_test/orders_landed.jsonl
```

**Expect:** `landed 124 records` (62 old + 62 new).

### Step 19 · Spark Job 1 with VM3 down start after: step 18

```
python spark/spark_job1.py --input outputs/failover_test/orders_landed.jsonl \
    --output outputs/failover_test/order_metrics.jsonl
```

**Expect:** still `enriched records written: 44`.

### Step 20 · Producer 2 with VM3 down start after: step 19

```
python producer/producer2.py --bootstrap-servers $B1 \
    --input outputs/failover_test/order_metrics.jsonl
```

**Expect:** `published 44 records`. Tell Member 1.

### Step 27 · Consumer 1 after recovery start after: Member 1, step 26

```
python consumer/consumer1.py --output outputs/recovery_test/orders_landed.jsonl
```

**Expect:** `landed 186 records`.

### Step 28 · Spark Job 1 after recovery start after: step 27

```
python spark/spark_job1.py --input outputs/recovery_test/orders_landed.jsonl \
    --output outputs/recovery_test/order_metrics.jsonl
```

**Expect:** `enriched records written: 44`.

### Step 29 · Producer 2 after recovery start after: step 28

```
python producer/producer2.py --input outputs/recovery_test/order_metrics.jsonl
```

**Expect:** `published 44 records`. Tell Member 1.

# 7. The failure test explained (whole team)

```
Before (both up):    orders P0  Leader: 2  Replicas: 2,1  Isr: 2,1
                     orders P1  Leader: 1  Replicas: 1,2  Isr: 1,2
After stopping VM3:  orders P0  Leader: 1  Replicas: 2,1  Isr: 1     <- leader moved to 1
                     orders P1  Leader: 1  Replicas: 1,2  Isr: 1
Restart VM3 -> Isr back to 2,1 / 1,2.
```

**Leader** = the broker currently serving that partition. **Replicas** = where copies are *supposed* to live (doesn't change). **Isr** = copies that are currently up to date (shrinks to just 1 while VM3 is off).

**Why the counts grow:** Kafka keeps old messages, so each run adds to the topics: orders 62 -> 124 -> 186, order_metrics 44 -> 88 -> 132. Those extra copies are exact duplicates, so Job 1 still outputs **44** and the CSVs are byte-identical (checked with `diff`).

# 8. Key words in one line each

| Term | Simple meaning |
|---|---|
| Broker | A Kafka server that stores messages (VM2, VM3). |
| Controller | The boss broker that tracks who is alive and picks leaders (only VM2). |
| KRaft | Kafka's built-in way of electing and storing the boss's records. It replaced ZooKeeper. |
| Topic | A named mailbox: *orders*, *order_metrics*. |
| Partition | A topic is split into pieces (we use 2) so work can be shared. Order is kept only inside one partition. |
| Offset | A message's position number inside a partition (0, 1, 2 ...). |
| Replication factor 2 | Every partition is stored twice, once on each broker. |
| Leader / follower | The leader copy handles reads and writes; the follower copies it. |
| ISR | In-Sync Replicas: the copies that are fully up to date. |
| acks=all | The producer gets "OK" only when all in-sync copies have the message. |
| Bootstrap servers | The first broker address(es) a client contacts to discover the whole cluster. |
| advertised.listeners | The address a broker tells clients to use. It must be the ZeroTier IP, not localhost. |
| ZeroTier | A virtual LAN, so the 4 laptops on different home networks can reach each other. |
| local[*] | Spark runs on this one machine, using all CPU cores. |
| Lazy evaluation | Spark only plans the work; it runs when an action (count, collect) is called. |

# 9. Top viva questions, short answers

### Q. Why use Kafka instead of just copying files?

It separates the stages (VM1 and VM4 never talk directly), stores the data reliably in two copies, and lets any stage re-read the data. The spec also forbids copying the file.

### Q. Why is Producer 1 unkeyed but Producer 2 keyed?

Raw lines may be broken JSON, so Producer 1 doesn't read them to find a key. Cleaned lines are always valid, so Producer 2 keys by order_id, which gives a fixed partition split (16/28).

### Q. Why doesn't the consumer use a consumer group?

Groups remember where they stopped, so a second run would read nothing. Ours is a batch job that must copy the whole topic every time, so it reads all partitions from offset 0 and commits nothing.

### Q. How does the consumer know when to stop?

When its position equals the last offset of every partition and nothing new has arrived for 10 s.

### Q. What happens when VM3 stops?

VM3 stops sending heartbeats, the controller on VM2 removes it from the ISR and makes broker 1 the leader everywhere. Broker 1 already has a full copy (RF 2), so no data is lost.

### Q. Why stop VM3 and not VM2?

VM2 is the only controller. Without it nobody can elect leaders. Real high availability needs 3 controllers.

### Q. Can you still produce while VM3 is down?

Yes. acks=all only waits for the in-sync copies (now just broker 1), and min.insync.replicas=1 allows that.

### Q. What if Producer 1 runs twice?

The topic has 124 messages, but Spark Job 1 still outputs 44 because duplicates are exact and get removed. Delivery is at-least-once, which is why both jobs dedupe.

### Q. Why collect() and write the CSV with Python?

Spark's writer makes a folder of part-files. The spec wants single files with exact names, and the results are tiny (6 and 4 rows).

### Q. Why is dropDuplicates in Job 2 if it removes 0?

The spec says both jobs must dedupe. It also protects against Kafka redelivery or a producer re-run.

### Q. Why not deduplicate by order_id?

The spec says *exact* duplicates. Choosing between two different records with the same ID would be inventing a business rule.

### Q. How do you know the outputs are correct?

tests/verify_outputs.py redoes everything in plain Python (no Spark) and compares every value. It prints "OK: 44 enriched records and both summaries match".

### Q. Why Spark for only 62 records?

The assignment is about the architecture. The same code would work on millions of rows or on a real cluster.

## 10. Numbers to remember

| Item | Value |
|---|---|
| Raw lines | 62 |
| Dropped by cleaning | 14 -> 48 valid |
| Exact duplicates removed | 4 -> 44 enriched |
| orders partition split | random, e.g. 31 / 31 |
| order_metrics partition split | always 16 / 28 |
| Restaurants / cities | 6 (R01-R06) / 4 (Bengaluru, Hyderabad, Mumbai, Pune) |
| Delayed orders | 33 of 44 (11 on time) |
| Delay categories | ON_TIME 11, SLIGHT 11, MODERATE 15, SEVERE 7 |
| Highest / lowest delay % | 100% (R02, R04, R06, Hyderabad) / 42.86% (R05, Pune) |
| Fastest / slowest delivery | R01 22.0 min / R06 47.0 min |
| Node IDs / ports | VM2 = node 1 (9092 + 9093); VM3 = node 2 (9092) |

| Item | Value |
| --- | --- |
| Versions | Kafka 4.3.1 (KRaft), PySpark 4.2.0, kafka-python 2.2.15, Java 17+ |